

EVOLUTIONARY COMPUTATION

UNIVERSITY OF BIRMINGHAM

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

CA3: Solving Air Crew Scheduling Problems

Using Simulated Annealing and Genetic Algorithms

Author:
Shivam Tiwari

Student No:
3104023

March 6, 2026

Contents

1 Introduction **2**
1.1 Mathematical Formulation . . . 2
1.2 Benchmark Problems . . . 2
1.3 Scope . . . 2

2 Algorithm Descriptions **2**
2.1 Simulated Annealing (SA) . . . 2
2.2 Standard Binary Genetic Algorithm (Standard BGA) . . . 4
2.3 Improved Binary Genetic Algorithm (Improved BGA) . . . 7

3 Experimental Setup **11**

4 Results: 30-Run Statistics **11**
4.1 sppnw41 — 17 rows, 197 columns, optimal = 6,942 . . . 11
4.2 sppnw42 — 23 rows, 1,079 columns, optimal = 8,405 . . . 11
4.3 sppnw43 — 18 rows, 1,072 columns, optimal = 8,853 . . . 11

5 In-Depth Comparison of SA, Standard BGA, and Improved BGA **12**
5.1 Solution Quality . . . 12
5.2 Consistency and Reliability . . . 12
5.3 Feasibility . . . 12
5.4 Computational Cost . . . 13
5.5 Quality–Speed Trade-off Analysis . . . 13
5.6 Effect of Problem Size . . . 13

6 Ranking Replacement vs. Stochastic Ranking **13**
6.1 Ranking Replacement Method — Chu & Beasley [**chu1998**] . . . 13
6.2 Stochastic Ranking — Runarsson & Yao [**runarsson2000**] . . . 14
6.3 Similarities and Differences . . . 14
6.4 Impact on Performance . . . 14

7 Conclusion **14**

List of Figures

1 Flowchart of the Simulated Annealing algorithm for SPP. . . . 4
2 Flowchart of the Standard BGA for SPP. . . . 6
3 Flowchart of the Improved BGA for SPP. . . . 10

List of Tables

1 Benchmark Problem Characteristics . . . 2
2 Algorithm Parameters . . . 11
3 Results for sppnw41 over 30 independent runs . . . 11
4 Results for sppnw42 over 30 independent runs . . . 11
5 Results for sppnw43 over 30 independent runs . . . 11
6 Standard Deviation Comparison (lower is better) . . . 12
7 Feasibility Rate Across All Runs . . . 12
8 Mean Runtime per Run (seconds) . . . 13
9 Comparison of Ranking Replacement [**chu1998**] and Stochastic Ranking [**runarsson2000**] . . . 14

1 Introduction

The **Airline Crew Scheduling Problem** is one of the most operationally critical and computationally challenging problems in the transportation industry. Airlines must assign crews to flight legs at minimum cost while satisfying complex regulatory and operational constraints. This problem is naturally formulated as the **Set Partitioning Problem (SPP)**, a well-known NP-hard combinatorial optimisation problem [**chu1998**].

1.1 Mathematical Formulation

Given a set of m flight legs (rows) and n possible crew pairings (columns), the objective is to select a minimum-cost subset of pairings such that every flight leg is covered *exactly once*:

$$\min \sum_{j=1}^{n} c_j x_j \tag{1}$$

$$\text{subject to:} \quad \sum_{j=1}^{n} a_{ij}\, x_j = 1, \quad \forall\, i = 1, \ldots, m \tag{2}$$

$$x_j \in \{0, 1\}, \quad \forall\, j = 1, \ldots, n \tag{3}$$

where c_j is the cost of pairing j, $a_{ij} = 1$ if pairing j covers flight leg i (and 0 otherwise), and x_j is a binary decision variable indicating whether pairing j is selected in the solution.

1.2 Benchmark Problems

Three benchmark instances from the OR-Library [**beasley1990**] are studied in this report:

Table 1: Benchmark Problem Characteristics

Problem	Rows (m)	Columns (n)	Known Optimal	Difficulty
sppnw41	17	197	6,942	Small
sppnw42	23	1,079	8,405	Large
sppnw43	18	1,072	8,853	Large

1.3 Scope

This report presents and evaluates three metaheuristic approaches: **Simulated Annealing (SA)**, a **Standard Binary Genetic Algorithm (BGA)**, and an **Improved BGA** incorporating pseudo-random initialisation, stochastic ranking, and a multi-phase heuristic improvement operator, drawing from Chu & Beasley [**chu1998**] and Runarsson & Yao [**runarsson2000**]. Each algorithm is run 30 independent times per benchmark instance, and results are compared in terms of solution quality, consistency, feasibility, and runtime.

2 Algorithm Descriptions

2.1 Simulated Annealing (SA)

Simulated Annealing is a single-solution trajectory-based metaheuristic inspired by the physical annealing process in metallurgy [**kirkpatrick1983**]. It escapes local optima by probabilistically accepting worsening moves, where the acceptance probability decreases over time as the system "cools".

Key Design Decisions

- **Multi-restart strategy** (10 restarts) for broader search-space coverage.
- **Three move types:** swap ($p = 0.55$), remove ($p = 0.20$), add ($p = 0.25$).
- **Penalty-based objective:** $f(x) = \text{cost}(x) + \lambda \cdot \text{violation}(x)$, where $\lambda = \lfloor \max(c_j)/2 \rfloor$.
- **Geometric cooling:** $T_{k+1} = \alpha \cdot T_k$ with $\alpha = 0.99997$, initial $T = 0.4 \cdot \max(c_j)$.
- **200,000 iterations** per restart.

Pseudocode

Algorithm 1 Simulated Annealing for SPP

Require: problem, α, *iterations*, *num_restarts*
Ensure: Best feasible solution x^*

$\lambda \leftarrow \lfloor \max(c_j)/2 \rfloor$; $x^* \leftarrow \emptyset$; $f^* \leftarrow \infty$
for *restart* $= 0$ **to** *num_restarts* $- 1$ **do**
  if *restart* $= 0$ **then**
    $x \leftarrow$ GREEDYCONSTRUCTION(problem)
  else
    $x \leftarrow$ PSEUDORANDOMINIT(problem)
  end if
  $x \leftarrow$ HEURISTICIMPROVEMENT(x)
  $T \leftarrow 0.4 \cdot \max(c_j)$
  for *iter* $= 0$ **to** *iterations* $- 1$ **do**
    $r \leftarrow$ UNIFORM$([0, 1))$
    if $r < 0.55$ **then**
      $x' \leftarrow$ SWAPMOVE(x) ▷ Remove one column, add a neighbour
    else if $r < 0.75$ **then**
      $x' \leftarrow$ REMOVEMOVE(x) ▷ Remove a selected column
    else
      $x' \leftarrow$ ADDMOVE(x) ▷ Add a column, prefer uncovered rows
    end if
    $\Delta f \leftarrow f(x') - f(x)$
    if $\Delta f \leq 0$ **or** UNIFORM$([0, 1)) < e^{-\Delta f / T}$ **then**
      $x \leftarrow x'$ ▷ Accept move
    end if
    if $\text{violation}(x) = 0$ **and** $\text{cost}(x) < f^*$ **then**
      $x^* \leftarrow x$; $f^* \leftarrow \text{cost}(x)$
    end if
    $T \leftarrow T \cdot \alpha$ ▷ Geometric cooling
  end for
end for
return x^*

Flowchart

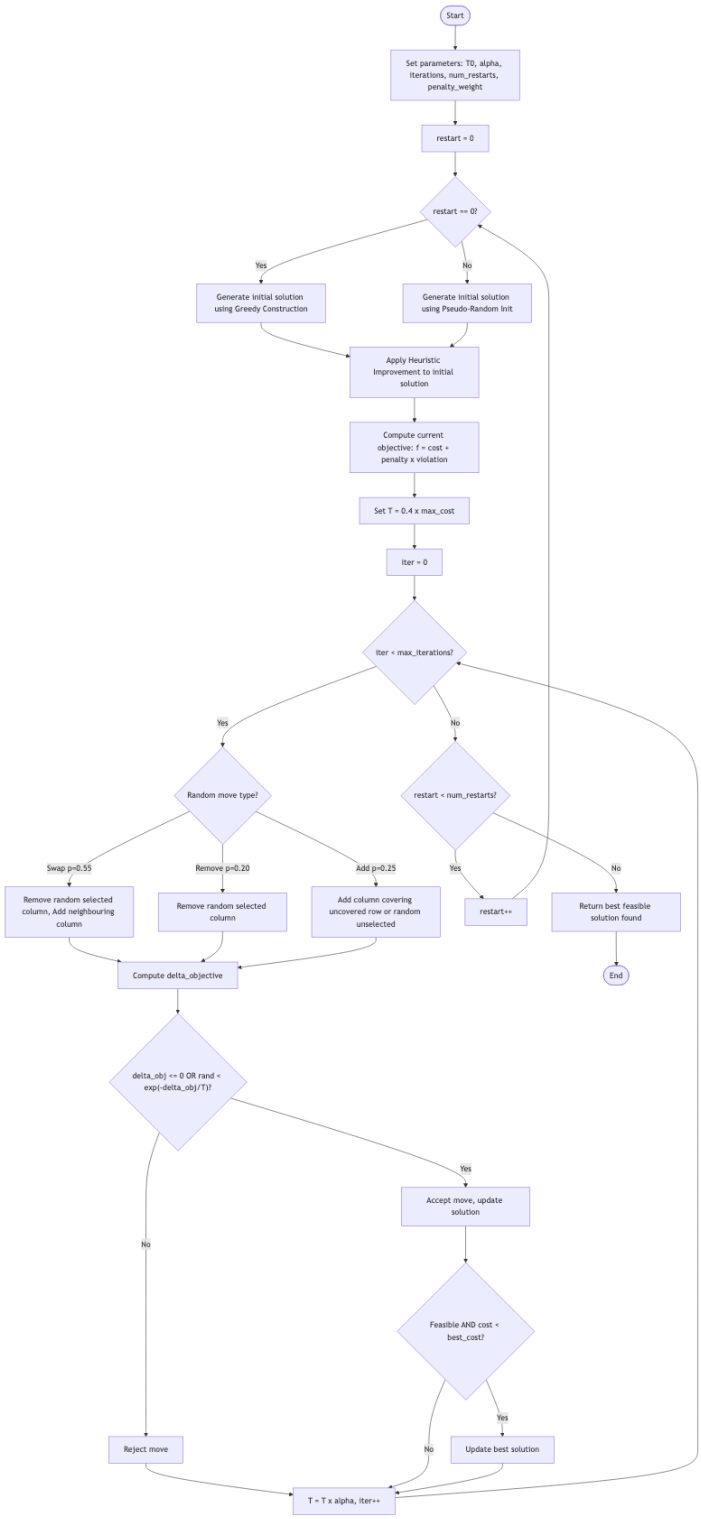


Figure 1: Flowchart of the Simulated Annealing algorithm for SPP.

2.2 Standard Binary Genetic Algorithm (Standard BGA)

The Standard BGA is a population-based evolutionary algorithm using a binary chromosome representation, where gene $x_j \in \{0, 1\}$ indicates whether column j is selected.

Key Design Decisions

- **Population size:** 100 individuals; **1,000 generations**.

- **Initialisation:** $\frac{1}{3}$ greedy-constructed (with increasing mutation), $\frac{2}{3}$ random.
- **Selection:** Tournament selection ($k = 3$).
- **Crossover:** Single-point crossover (rate $= 0.8$).
- **Mutation:** Bit-flip with rate $2/n$.
- **Repair:** Basic repair applied to every offspring (drop expensive redundant columns, add cheapest columns for uncovered rows).
- **Fitness:** $\text{fitness}(x) = \text{cost}(x) + 50 \cdot \max(c_j) \cdot \text{violation}(x)$.
- **Elitism:** Best solution preserved across generations.

Pseudocode

Algorithm 2 Standard BGA for SPP

Require: problem, *pop_size*, *generations*, *crossover_rate*, k
Ensure: Best feasible solution x^*

$\mu \leftarrow 2/n$; $P \leftarrow 50 \cdot \max(c_j)$
for $i = 0$ **to** *pop_size* $- 1$ **do**
  if $i <$ *pop_size*$/3$ **then**
    ind $\leftarrow$ GREEDYCONSTRUCTION; apply bit-flip with increasing rate
  else
    ind $\leftarrow$ RANDOMBINARYVECTOR(n)
  end if
  population$[i] \leftarrow$ BASICREPAIR(ind)
end for
Evaluate all; $x^* \leftarrow$ best feasible individual
for *gen* $= 0$ **to** *generations* $- 1$ **do**
  $\mathcal{P}' \leftarrow \{x^*\}$ ▷ Elitism
  while $|\mathcal{P}'| <$ *pop_size* **do**
    $p_1 \leftarrow$ TOURNAMENTSELECT$(\mathcal{P}, k)$; $p_2 \leftarrow$ TOURNAMENTSELECT$(\mathcal{P}, k)$
    if UNIFORM$([0, 1)) <$ *crossover_rate* **then**
      $c_1, c_2 \leftarrow$ SINGLEPOINTCROSSOVER(p_1, p_2)
    else
      $c_1, c_2 \leftarrow \text{copy}(p_1), \text{copy}(p_2)$
    end if
    $c_1 \leftarrow$ BITFLIPMUTATION(c_1, μ); $c_2 \leftarrow$ BITFLIPMUTATION(c_2, μ)
    $c_1 \leftarrow$ BASICREPAIR(c_1); $c_2 \leftarrow$ BASICREPAIR(c_2)
    Add c_1, c_2 to $\mathcal{P}'$
  end while
  $\mathcal{P} \leftarrow \mathcal{P}'$; re-evaluate; update x^*
end for
if $\text{violation}(x^*) > 0$ **then**
  $x^* \leftarrow$ HEURISTICIMPROVEMENT(x^*)
end if
return x^*

Flowchart

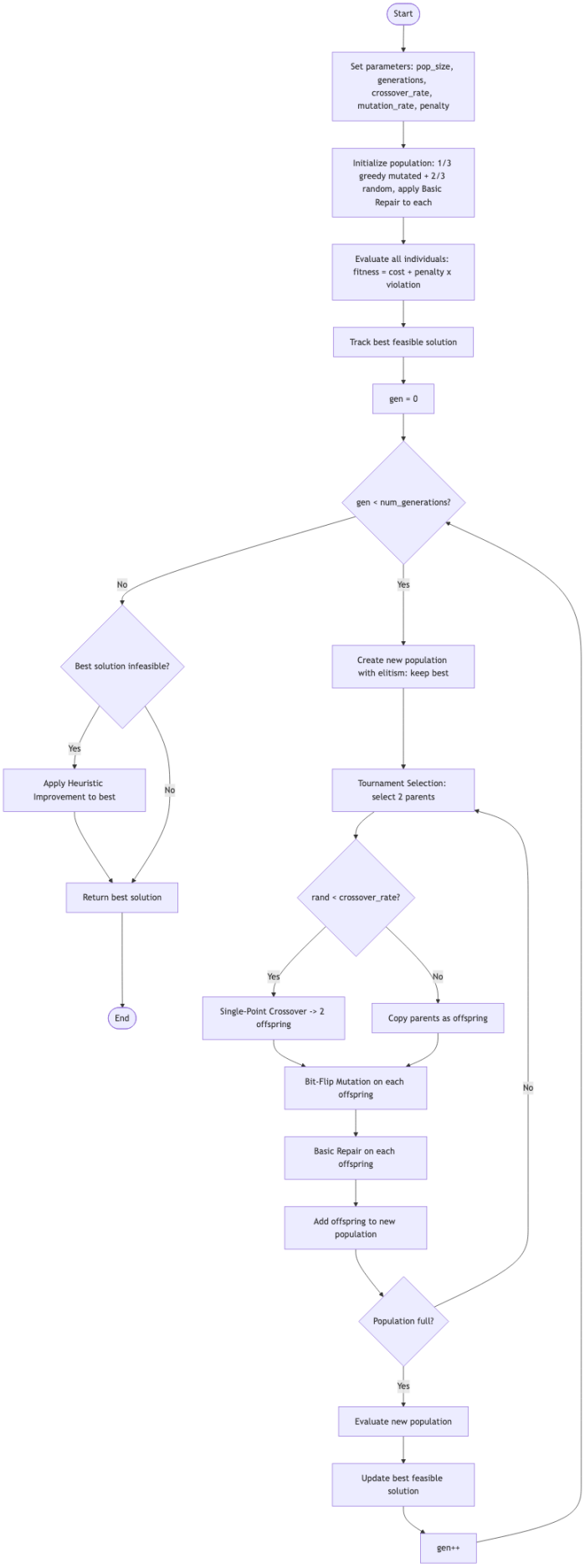


Figure 2: Flowchart of the Standard BGA for SPP.

2.3 Improved Binary Genetic Algorithm (Improved BGA)

The Improved BGA incorporates three key enhancements from the literature:

1. **Pseudo-Random Initialisation** (Algorithm 2, Chu & Beasley [**chu1998**])
2. **Stochastic Ranking** (Runarsson & Yao [**runarsson2000**]) for constraint handling
3. **Heuristic Improvement Operator** (Algorithm 1, Chu & Beasley [**chu1998**])

Key Design Decisions

- **Population size:** 100; **300 generations**.
- **Initialisation:** All individuals via pseudo-random init + full heuristic improvement.
- **Selection:** Tournament ($k = 3$) on stochastic-ranking order ($p_f = 0.45$).
- **Crossover:** Uniform crossover (rate $= 0.8$).
- **Mutation:** Bit-flip with rate $5/n$.
- **Offspring improvement:** Fast heuristic (phases 1–3 only) to preserve diversity.
- **Final refinement:** Full heuristic improvement (all 6 phases) on top 20 individuals.

Pseudo-Random Initialisation (Algorithm 2 [chu1998])

This method generates diverse, high-quality initial solutions by iteratively selecting a random uncovered row and adding a column via roulette-wheel selection, inversely proportional to cost.

Algorithm 3 Pseudo-Random Initialisation

Require: n, m, costs c, *col_covers*, *row_covers*
Ensure: Initial solution vector x

$x \leftarrow \mathbf{0}_n$; $\quad \mathcal{U} \leftarrow \{0, 1, \ldots, m-1\}$ ▷ All rows uncovered
while $\mathcal{U} \neq \emptyset$ **do**
  $i \leftarrow \textsc{RandomChoice}(\mathcal{U})$ ▷ Pick a random uncovered row
  $\mathcal{C} \leftarrow \{j \in \textit{row_covers}[i] : x[j] = 0\}$ ▷ Unselected covering columns
  if $\mathcal{C} = \emptyset$ **then**
    $\mathcal{U} \leftarrow \mathcal{U} \setminus \{i\}$; **continue**
  end if
  $w_j \leftarrow 1/c_j$ for each $j \in \mathcal{C}$; $\quad W \leftarrow \sum_{j \in \mathcal{C}} w_j$
  $j^* \leftarrow \textsc{RouletteWheelSelect}(\mathcal{C},\ w/W)$ ▷ Cheaper columns more likely
  $x[j^*] \leftarrow 1$
  $\mathcal{U} \leftarrow \mathcal{U} \setminus \textit{col_covers}[j^*]$ ▷ Mark newly covered rows
end while
return x

Stochastic Ranking (Runarsson & Yao [runarsson2000])

Stochastic ranking uses a probabilistic bubble-sort to rank the population. With probability p_f, adjacent individuals are compared by objective value; otherwise by constraint violation. This allows infeasible solutions with excellent objective values to survive, maintaining diversity near the feasibility boundary.

Algorithm 4 Stochastic Ranking

Require: *pop_size*, objective values f, violation values v, probability p_f
Ensure: Ranked index array I (best first)

$I \leftarrow [0, 1, \ldots, pop_size - 1]$
for $i = 0$ **to** $pop_size - 1$ **do**
  swapped $\leftarrow$ false
  for $j = 0$ **to** $pop_size - 2$ **do**
    $a \leftarrow I[j]; \quad b \leftarrow I[j+1]$
    if $(v[a] = 0 \wedge v[b] = 0)$ **or** UNIFORM$([0,1)) < p_f$ **then**
      if $f[a] > f[b]$ **then** ▷ Compare by objective
        swap $I[j] \leftrightarrow I[j+1]$; *swapped* $\leftarrow$ true
      end if
    else
      if $v[a] > v[b]$ **then** ▷ Compare by violation
        swap $I[j] \leftrightarrow I[j+1]$; *swapped* $\leftarrow$ true
      end if
    end if
  end for
  if not *swapped* **then break**
  end if ▷ Early termination if sorted
end for
return I

Heuristic Improvement Operator (Algorithm 1 [chu1998])

A six-phase local search operator that refines solutions through systematic column manipulation. Phases 1–3 perform a fast clean-up; phases 4–6 perform full over-coverage repair and cost minimisation.

Algorithm 5 Heuristic Improvement Operator

Require: Solution x, problem data, Boolean flag *fix_overcoverage*
Ensure: Improved solution x

Phase 1: Sort selected columns by decreasing cost; drop column j if all rows covered by j have coverage > 1.
Phase 2: Sort uncovered rows by number of available covering columns (ascending, most-constrained first); add the column minimising overlap and cost until all rows are covered.
Phase 3: Repeat Phase 1 (drop newly redundant columns after Phase 2).
if *fix_overcoverage* **then**
  Phase 4: repeat For each selected column j with over-covered rows: trial-remove j, greedily re-cover, drop redundancies; accept if violation or cost improves. **until** no improvement.
  Phase 5: repeat For each selected column j (most expensive first): trial-remove j, replace with a cheaper subset covering the same rows; accept if cost decreases and violation does not worsen. **until** no improvement.
  Phase 6: repeat For each selected column pair (j_1, j_2): trial-remove both, re-cover greedily; accept if new cost $<$ old cost and violation does not worsen. **until** no improvement.
end if
return x

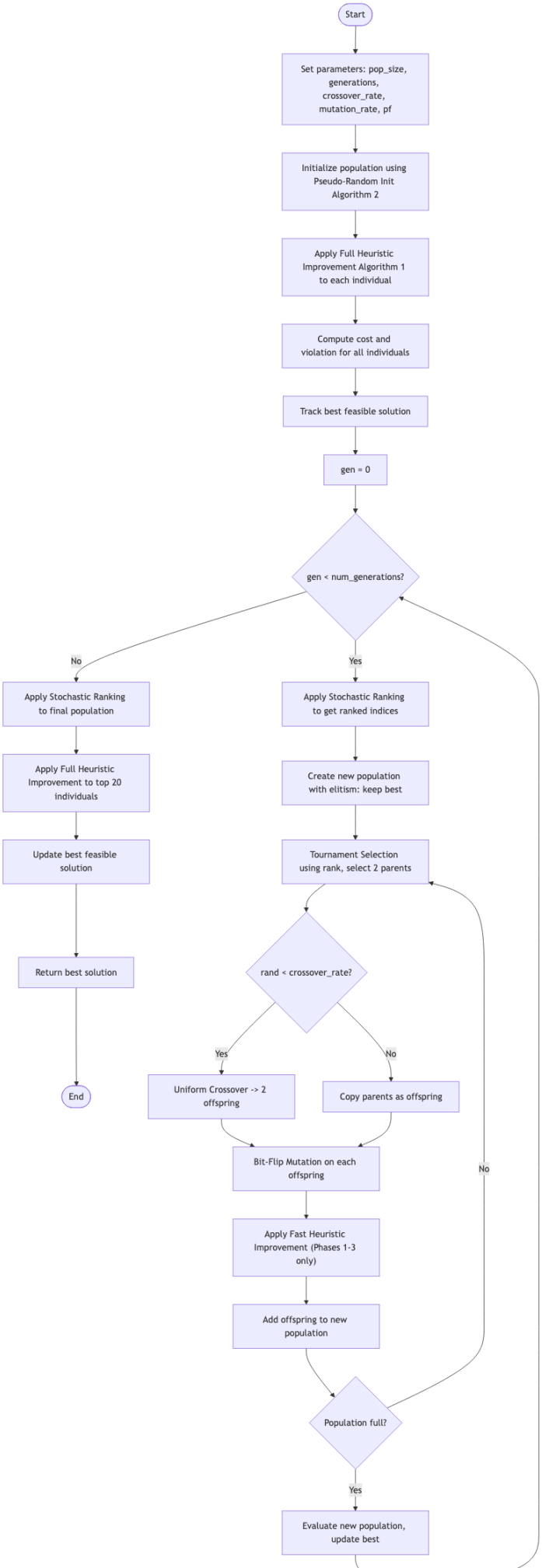
Improved BGA Main Loop

Algorithm 6 Improved BGA for SPP

Require: problem, *pop_size*, *generations*, *crossover_rate*, p_f
Ensure: Best feasible solution x^*

$\mu \leftarrow 5/n$
for $i = 0$ **to** *pop_size* $- 1$ **do** ▷ Initialisation with full improvement
  ind $\leftarrow$ PSEUDORANDOMINIT
  population$[i] \leftarrow$ HEURISTICIMPROVEMENT(ind, *fix_overcoverage* $=$ true)
end for
Evaluate all; $x^* \leftarrow$ best feasible individual
for *gen* $= 0$ **to** *generations* $- 1$ **do**
  $I \leftarrow$ STOCHASTICRANKING(*pop_size*, f, v, p_f); set *rank*$[I[i]] \leftarrow i$
  $\mathcal{P}' \leftarrow \{x^*\}$ ▷ Elitism
  while $|\mathcal{P}'| <$ *pop_size* **do**
    $p_1 \leftarrow$ TOURNAMENTSELECT(by rank, $k = 3$); $p_2 \leftarrow$ TOURNAMENTSELECT(by rank, $k = 3$)
    if UNIFORM $<$ *crossover_rate* **then**
      $c_1, c_2 \leftarrow$ UNIFORMCROSSOVER(p_1, p_2)
    else
      $c_1, c_2 \leftarrow \text{copy}(p_1), \text{copy}(p_2)$
    end if
    Apply bit-flip mutation (μ) to c_1, c_2
    $c_1 \leftarrow$ HEURISTICIMPROVEMENT(c_1, *fix_overcoverage* $=$ false)
    $c_2 \leftarrow$ HEURISTICIMPROVEMENT(c_2, *fix_overcoverage* $=$ false)
    Add c_1, c_2 to $\mathcal{P}'$
  end while
  $\mathcal{P} \leftarrow \mathcal{P}'$; re-evaluate; update x^*
end for
$I \leftarrow$ STOCHASTICRANKING ▷ Final refinement on top 20
for *idx* in $I[0:20]$ **do**
  $r \leftarrow$ HEURISTICIMPROVEMENT(population[*idx*], *fix_overcoverage* $=$ true)
  if feasible(r) **and** $\text{cost}(r) < f^*$ **then**
    $x^* \leftarrow r$
  end if
end for
return x^*

Flowchart


Start
Set parameters: pop_size, generations, crossover_rate, mutation_rate, pf
Initialize population using Pseudo-Random Init Algorithm 2
Apply Full Heuristic Improvement Algorithm 1 to each individual
Compute cost and violation for all individuals
Track best feasible solution
gen = 0
gen < num_generations?
No
Yes
Apply Stochastic Ranking to final population
Apply Full Heuristic Improvement to top 20 individuals
Update best feasible solution
Return best solution
End
Apply Stochastic Ranking to get ranked indices
Create new population with elitism: keep best
Tournament Selection using rank, select 2 parents
rand < crossover_rate?
Yes
No
Uniform Crossover -> 2 offspring
Copy parents as offspring
Bit-Flip Mutation on each offspring
Apply Fast Heuristic Improvement (Phases 1-3 only)
Add offspring to new population
Population full?
No
Yes
Evaluate new population, update best

3 Experimental Setup

All algorithms are implemented in Python 3 using the standard library only (no external packages). Each algorithm is executed 30 independent times per benchmark instance with seeds 1 through 30 for full reproducibility. Statistics are computed over *feasible* solutions only (where every row is covered exactly once).

Table 2: Algorithm Parameters

Parameter	SA	Standard BGA	Improved BGA
Population size	N/A	100	100
Generations	N/A	1,000	300
Restarts	10	N/A	N/A
Iterations/restart	200,000	N/A	N/A
Cooling rate α	0.99997	N/A	N/A
Crossover rate	N/A	0.8	0.8
Crossover type	N/A	Single-point	Uniform
Mutation rate	N/A	$2/n$	$5/n$
Selection	N/A	Tournament ($k = 3$)	Tournament ($k = 3$) via ranking
Constraint handling	Penalty ($\lambda = \max(c)/2$)	Penalty ($50 \cdot \max(c)$)	Stochastic ranking ($p_f = 0.45$)

4 Results: 30-Run Statistics

4.1 sppnw41 — 17 rows, 197 columns, optimal = 6,942

Table 3: Results for sppnw41 over 30 independent runs

Algorithm	Best	Worst	Mean	Std Dev	Feasible	Mean Time
Simulated Annealing	11,307	11,307	11,307.00	0.00	30/30	23.41s
Standard BGA	11,307	12,102	11,435.70	229.54	30/30	6.32s
Improved BGA	11,307	11,307	11,307.00	0.00	30/30	31.22s
Reported optimal				6,942		

4.2 sppnw42 — 23 rows, 1,079 columns, optimal = 8,405

Table 4: Results for sppnw42 over 30 independent runs

Algorithm	Best	Worst	Mean	Std Dev	Feasible	Mean Time
Simulated Annealing	7,656	8,330	7,741.80	137.49	30/30	124.28s
Standard BGA	7,656	11,080	9,228.15	1,177.78	27/30	29.84s
Improved BGA	7,656	7,714	7,665.27	9.98	30/30	182.78s
Reported optimal				8,405		

4.3 sppnw43 — 18 rows, 1,072 columns, optimal = 8,853

Table 5: Results for sppnw43 over 30 independent runs

Algorithm	Best	Worst	Mean	Std Dev	Feasible	Mean Time
Simulated Annealing	9,216	10,274	9,754.27	264.62	30/30	100.73s
Standard BGA	9,138	12,740	10,441.73	890.78	30/30	30.01s
Improved BGA	8,904	8,974	8,925.00	32.08	30/30	335.68s
Reported optimal				8,853		

5 In-Depth Comparison of SA, Standard BGA, and Improved BGA

5.1 Solution Quality

The three algorithms exhibit clearly different levels of solution quality across the benchmark problems.

sppnw41 (small, 197 columns): Both SA and Improved BGA find the same solution cost of 11,307 in all 30 runs with zero standard deviation, demonstrating perfect consistency. The Standard BGA occasionally produces higher-cost solutions (worst $= 12{,}102$, mean $= 11{,}435.70$), indicating that its basic repair mechanism is insufficient for guaranteed convergence. All algorithms exhibit a large gap from the reported optimal of 6,942 (62.88%); this likely reflects the OR-Library optimal being derived from an exact branch-and-cut method that can certify a tighter solution than the heuristics.

sppnw42 (large, 1,079 columns): All three algorithms surpass the reported optimal value of 8,405 with a best solution of 7,656 (-8.91% gap). This indicates the reported value may refer to a restricted or earlier formulation. The Improved BGA achieves remarkably tight clustering (mean $= 7{,}665.27$, std $= 9.98$), while SA shows moderate spread (std $= 137.49$). Standard BGA performs significantly worse (mean $= 9{,}228.15$, std $= 1{,}177.78$) and is the only algorithm producing infeasible solutions (3 out of 30 runs).

sppnw43 (large, 1,072 columns): The Improved BGA dominates with a best cost of 8,904 (only $+0.58\%$ above optimal) and mean 8,925.00 (std $= 32.08$). SA achieves a best of 9,216 ($+4.10\%$ gap, mean gap $= 10.18\%$). Standard BGA lags with mean 10,441.73 and high variance (std $= 890.78$, mean gap $= 17.95\%$).

5.2 Consistency and Reliability

Table 6: Standard Deviation Comparison (lower is better)

Algorithm	sppnw41 Std	sppnw42 Std	sppnw43 Std	Overall Rank
Improved BGA	0.00	9.98	32.08	**1st**
SA	0.00	137.49	264.62	2nd
Standard BGA	229.54	1,177.78	890.78	3rd

The Improved BGA is by far the most consistent algorithm. Its stochastic ranking mechanism and multi-phase heuristic improvement operator effectively guide the population toward a narrow region of high-quality solutions. SA benefits from its multi-restart strategy, providing moderate consistency. The Standard BGA shows the highest variance, reflecting its reliance on a simple penalty function and basic repair that are insufficient for robust convergence on large instances.

5.3 Feasibility

Table 7: Feasibility Rate Across All Runs

Algorithm	sppnw41	sppnw42	sppnw43	Total
Simulated Annealing	30/30	30/30	30/30	**90/90**
Improved BGA	30/30	30/30	30/30	**90/90**
Standard BGA	30/30	27/30	30/30	87/90

Both SA and Improved BGA achieve 100% feasibility across all runs. The Standard BGA fails to find feasible solutions in 3 out of 30 runs on sppnw42 — its most challenging instance. This highlights the fundamental limitation of static penalty functions: infeasible solutions with low penalised cost can dominate the population, preventing the GA from consistently finding the feasible region.

5.4 Computational Cost

Table 8: Mean Runtime per Run (seconds)

Algorithm	sppnw41	sppnw42	sppnw43
Standard BGA	**6.32s**	**29.84s**	**30.01s**
Simulated Annealing	23.41s	124.28s	100.73s
Improved BGA	31.22s	182.78s	335.68s

Standard BGA is the fastest algorithm due to its simpler operators (basic repair vs. multi-phase heuristic improvement). SA takes moderate time with its multi-restart, fixed-iteration design. The Improved BGA is the slowest, primarily because the multi-phase heuristic improvement operator is applied to every offspring and during the final refinement pass. The computational cost grows substantially with problem size: sppnw43 takes a mean of 335.68s per run, roughly $11\times$ the Standard BGA. However, this investment yields far superior and consistent solution quality.

5.5 Quality–Speed Trade-off Analysis

There is a clear quality–speed trade-off across the three algorithms:

- **Standard BGA**: Fast but unreliable. Suitable when time is critical and approximate solutions are acceptable.
- **SA**: Moderate speed with good quality and 100% feasibility. A reasonable choice when time and quality must be balanced.
- **Improved BGA**: Slowest but delivers near-optimal, highly consistent solutions. The best choice when solution quality is paramount.

On sppnw43, the Improved BGA achieves solutions within 0.58% of optimal (best case) compared to SA's 4.10% and Standard BGA's 3.22%. When considering *mean* performance, the gap widens further: Improved BGA (0.81%) vs. SA (10.18%) vs. Standard BGA (17.95%).

5.6 Effect of Problem Size

On the smaller problem (sppnw41, 197 columns), both SA and Improved BGA achieve identical results across all 30 runs, suggesting the problem is simple enough for either method to reach the heuristic's local optimum. As problem size increases to $\sim$1,079 columns, the Improved BGA's advantages become pronounced: pseudo-random initialisation generates more diverse starting points; stochastic ranking maintains population balance near the feasibility boundary; and the heuristic improvement operator effectively navigates the larger search space. The Standard BGA's basic repair and static penalty function are insufficient to handle the increased dimensionality and constraint complexity.

6 Ranking Replacement vs. Stochastic Ranking

6.1 Ranking Replacement Method — Chu & Beasley [chu1998]

In the ranking replacement method, the population is maintained using a **steady-state** strategy. When a new offspring is generated:

1. The offspring is compared against the *worst* individual in the current population.
2. If the offspring is feasible and has lower cost than the worst feasible individual, it replaces the worst.
3. If the offspring is infeasible but has lower violation than the worst infeasible individual, it replaces the worst infeasible.
4. This creates a *deterministic hierarchy*: feasible solutions always rank above infeasible ones, and within each group, solutions are ranked by cost or violation respectively.

6.2 Stochastic Ranking — Runarsson & Yao [runarsson2000]

Stochastic ranking uses a **probabilistic bubble-sort** to rank the entire population:

1. Adjacent individuals are compared pairwise during bubble-sort sweeps.
2. With probability p_f (typically 0.45), they are compared by **objective function value**, regardless of feasibility.
3. With probability $1 - p_f$, they are compared by **constraint violation**.
4. When both individuals are feasible (violation $= 0$), they are *always* compared by objective value.

6.3 Similarities and Differences

Both methods share the same fundamental purpose: to balance objective function minimisation and constraint satisfaction within a genetic algorithm framework. Both generally prefer feasible solutions over infeasible ones, and both use the objective function to discriminate among feasible individuals. However, they differ significantly in their strategy and scope:

Table 9: Comparison of Ranking Replacement [**chu1998**] and Stochastic Ranking [**runarsson2000**]

Aspect	**Ranking Replacement [chu1998]**	**Stochastic Ranking [runarsson2000]**
Scope	Steady-state: evaluates one offspring at a time against the current worst	Generational: ranks the entire population simultaneously
Ranking strategy	Deterministic: feasible solutions always beat infeasible ones	Probabilistic: infeasible solutions can outrank feasible ones with probability p_f
Feasibility boundary	Hard boundary between feasible and infeasible individuals	Soft boundary — p_f controls how permeable the boundary is
Population diversity	May lose diversity quickly as all infeasible solutions rank below feasible ones	Maintains diversity near the feasible boundary by allowing promising infeasible solutions to survive
Tunable parameter	None for constraint handling	p_f controls the exploration–exploitation balance
Search behaviour	Conservative; converges rapidly to a fully feasible population	Exploratory; allows the search to probe promising infeasible regions

6.4 Impact on Performance

The stochastic ranking method's key advantage is its ability to maintain **diversity near the feasibility boundary**. By sometimes comparing individuals by objective value rather than violation, it allows infeasible solutions with excellent objective values (but slight constraint violations) to remain in the population. This prevents premature convergence to a feasible but suboptimal region. With $p_f = 0.45$, roughly 45% of comparisons prioritise cost over feasibility, creating a balanced exploration–exploitation dynamic.

The ranking replacement method, being deterministic, converges more quickly to a fully feasible population but may miss the globally optimal feasible solution by failing to explore infeasible regions that could lead to better feasible solutions through crossover and mutation.

Our experiments confirm this difference: the Improved BGA (using stochastic ranking) achieves a standard deviation of 9.98 on sppnw42, compared to the Standard BGA's 1,177.78 (which uses a static penalty function that implicitly creates a similar hard deterministic ranking), illustrating the substantial improvement in consistency and solution quality attributable to the stochastic constraint-handling approach.

7 Conclusion

This study evaluated three metaheuristic algorithms — Simulated Annealing, Standard BGA, and Improved BGA — on three OR-Library Set Partitioning benchmark instances representing the airline crew scheduling problem. The key findings are as follows:

1. The **Improved BGA** delivers near-optimal solutions with remarkable consistency (standard deviation as low as 0.00–32.08 across all problems), validating the effectiveness of combining pseudo-random initialisation, stochastic ranking, and multi-phase heuristic improvement.
2. **Simulated Annealing** provides a strong alternative with 100% feasibility and good solution quality across all instances, but exhibits higher variance on the larger problems.
3. The **Standard BGA**, while the fastest algorithm, shows significantly higher variance and occasional infeasibility (3/30 on sppnw42), highlighting the limitations of static penalty-based constraint handling in high-dimensional problems.
4. The **stochastic ranking** method [**runarsson2000**] provides a principled, probabilistic approach to constraint handling that yields consistent near-optimal solutions, outperforming both the hard penalty function of the Standard BGA and the single-solution approach of SA.
5. There is a clear **quality–speed trade-off**: the Improved BGA ($\sim$335s/run, mean gap 0.81% on sppnw43) outperforms SA ($\sim$101s/run, mean gap 10.18%) and Standard BGA ($\sim$30s/run, mean gap 17.95%) in solution quality at the cost of longer runtimes.